

Memória virtual, memória compartilhada e comunicação entre processos

Disciplina: Sistemas Operacionais

Professor: Michael Douglas C Alves

Alunos: Vitor Hugo, Thiago Wyse e Matheus Bertemes

Implementacao: <https://github.com/thiagowyseoficial/atividade-so-collatz>

Data: 01/10/2026

1. Objetivo

Implementar dois processos independentes em C++ no Windows para demonstrar a comunicação por memória compartilhada. O produtor recebe um inteiro positivo, calcula a sequência de Collatz e publica seus valores. O consumidor lê essa sequência e a apresenta, sem recalculá-la. A sincronização deve impedir a leitura de dados incompletos.

2. Fundamentação teórica

Um programa é um conjunto de instruções armazenado; um processo é uma instância em execução, com recursos, estado e espaço de endereçamento. Duas instâncias de um programa podem executar o mesmo código e manter dados privados diferentes.

Memória virtual é a abstração que separa os endereços usados pelo programa da organização da memória física. Ela permite proteção, isolamento, compartilhamento controlado e gerenciamento flexível dos dados residentes na RAM. Não se resume ao uso de disco como extensão da RAM.

Na paginação, o espaço virtual é dividido em páginas, enquanto a memória física é organizada em molduras, ou frames. A tradução de endereços relaciona páginas virtuais a molduras físicas. Uma página necessária que não está residente pode ocasionar uma falta de página, tratada pelo sistema operacional; isso não implica necessariamente acesso a disco.

IPC é a comunicação entre processos. A memória compartilhada permite que regiões dos espaços virtuais de processos diferentes se refiram ao mesmo armazenamento. As demais regiões continuam privadas. A documentação da Microsoft descreve a tradução por tabelas de páginas e a natureza privada do espaço de cada processo [1].

A Conjectura de Collatz propõe que, partindo de qualquer inteiro positivo, a repetição de $n/2$ para valores pares e $3n+1$ para ímpares alcança 1. O programa implementa essas regras, mas não

demonstra a conjectura. O argumento é o valor inicial, não a quantidade de elementos.

3. Implementação

3.1 Organização dos dados e limites

O arquivo `compartilhado.h` define a estrutura usada pelos dois programas:

```
struct DadosCompartilhados {
    std::uint64_t quantidade;
    std::uint64_t valores[4096];
};
```

O tamanho lógico solicitado é $8 + 4096 \times 8 = \mathbf{32.776 \text{ bytes}}$. Um `static_assert` verifica esse tamanho na compilação. O gerenciamento físico trabalha com páginas; esse número não representa necessariamente a quantidade exata de RAM residente. A capacidade fixa mantém o protocolo simples e evita escrever além do mapeamento.

O campo `quantidade` registra o número efetivo de elementos, incluindo o inicial e o 1 final. O consumidor verifica se esse campo está entre 1 e 4096 antes de percorrer o vetor. Os elementos não usados são ignorados. Não existem ponteiros dentro da estrutura compartilhada.

O tipo `uint64_t` representa valores de 0 a 18.446.744.073.709.551.615. A entrada aceita somente dígitos e deve ser maior que zero. O produtor verifica o limite durante a conversão e antes de calcular $3n+1$. Se houver overflow ou mais de 4096 elementos, a operação termina com erro antes de publicar dados. Assim, nem toda entrada positiva representável necessariamente produz uma sequência aceita.

3.2 Produtor

Primeiro valida o argumento e calcula a sequência em um vetor privado. Em seguida cria dois eventos nomeados e um objeto de mapeamento. `CreateFileMappingW` recebe `INVALID_HANDLE_VALUE`, de modo que o objeto tenha respaldo no arquivo de paginação do sistema, sem utilizar um arquivo de dados como canal de comunicação [2].

Depois de `MapViewOfFile`, o produtor copia o vetor para a vista compartilhada e grava a quantidade. Somente após terminar essas escritas executa `SetEvent(pronto)`. Ele aguarda a confirmação de leitura antes de liberar seus recursos. Ambos os programas mostram o PID e o endereço virtual da vista para facilitar a observação em sala.

3.3 Consumidor

O consumidor abre o objeto pelo mesmo nome com `OpenFileMappingW` e cria uma vista somente de leitura. Abre os eventos usando `OpenEventW` e aguarda o evento de dados prontos. Depois lê a quantidade e os valores, imprime a sequência e sinaliza a confirmação. Não implementa as regras de Collatz.

3.4 Protocolo de sincronização e encerramento

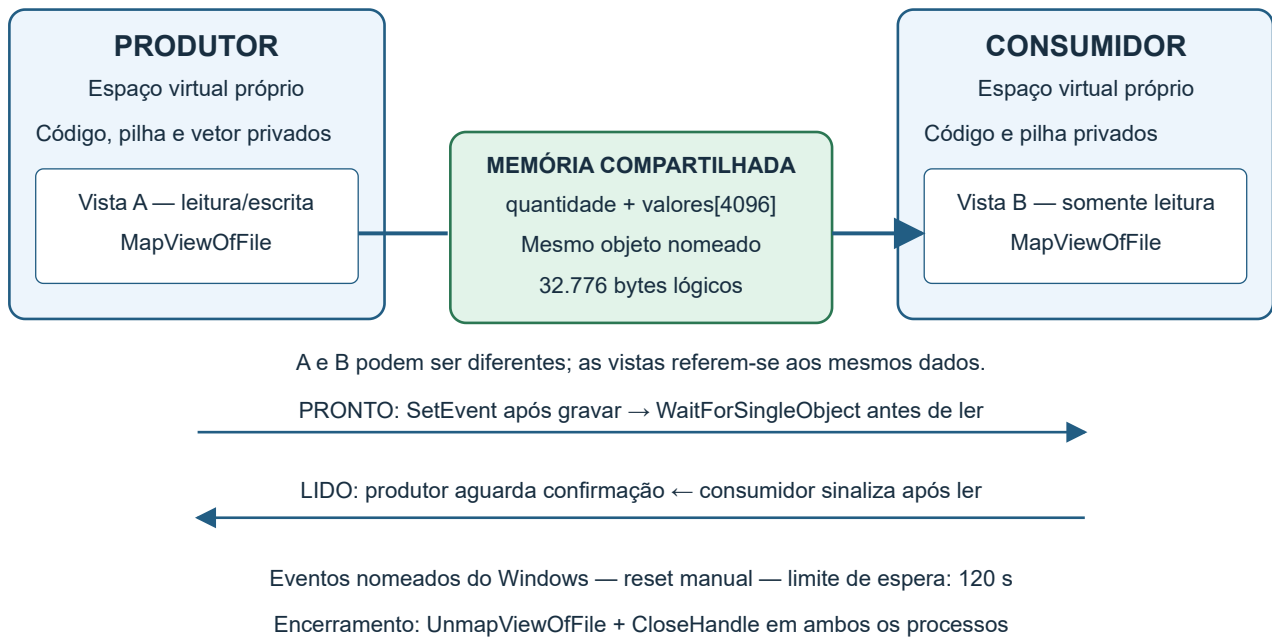
São usados dois eventos de reset manual, inicialmente não sinalizados: `Pronto`, do produtor para o consumidor, e `Lido`, no sentido inverso. O primeiro impede leitura antecipada; o segundo mantém o produtor vivo até a confirmação. Como o evento manual permanece sinalizado, o consumidor também funciona quando começa a esperar depois da publicação [3].

As operações de sinalização e espera do Windows fornecem as barreiras adequadas de ordenação de memória [4]. Um atraso artificial ou uma variável booleana comum não substitui esse protocolo. Não há escrita adicional no bloco depois de sua publicação, portanto não é necessário um mutex para o acesso ao vetor nesta solução de publicação única.

Os nomes usam o prefixo `Local\`, para a sessão atual. O projeto admite um produtor e um consumidor por rodada. O produtor rejeita objetos existentes para não misturar rodadas. Os eventos são criados antes do mapeamento; assim, quando o consumidor consegue abrir o mapeamento, os eventos já foram criados.

Cada espera tem limite de 120 segundos. Timeout ou falha da API encerra o programa com código 1. O uso de RAII chama `UnmapViewOfFile` e `CloseHandle` ao sair dos respectivos escopos, inclusive após exceções. Encerrar todos os participantes permite ao Windows liberar os objetos; o encerramento de um processo não invalida automaticamente a vista que outro ainda mantém.

4. Diagrama da arquitetura



As vistas podem começar em endereços virtuais diferentes. Os endereços do diagrama são simbólicos: A e B não são endereços físicos e não precisam coincidir. O fluxo dos dados é produtor → memória compartilhada → consumidor; os eventos controlam quando ler e quando encerrar.

5. Compilação, execução e testes

5.1 Compilação realizada

Os fontes foram compilados no Windows x64 com o compilador MSVC instalado em 14.51.36231, a partir do Visual Studio 2026 Developer Command Prompt v18.7.3. Comandos utilizados:

```
cl /nologo /std:c++17 /EHsc /W4 /WX /O2 /MT /utf-8 produtor.cpp /Fe:produtor.exe
cl /nologo /std:c++17 /EHsc /W4 /WX /O2 /MT /utf-8 consumidor.cpp /Fe:consumidor.exe
```

A compilação terminou com código 0 e sem avisos; /WX trata avisos como erros. O cabeçalho compartilhado.h deve acompanhar os dois arquivos CPP.

5.2 Limitação encontrada na execução

A tentativa de iniciar os executáveis foi bloqueada pela política de controle de aplicativos do Windows. A execução direta informou: “An Application Control policy has blocked this file”.

Portanto, **não foi possível concluir os testes de execução neste ambiente**. Compilação bem-sucedida não comprova o funcionamento em tempo de execução.

5.3 Três testes obrigatórios — resultados esperados

Em cada rodada, executar o produtor no primeiro terminal e, após a indicação de dados prontos, executar o consumidor no segundo terminal. Os dois devem encerrar antes da rodada seguinte.

Teste	Terminal 1	Terminal 2	Sequência esperada no consumidor	Quantidade
1	produtor.exe 5	consumidor.exe	5 16 8 4 2 1	6
2	produtor.exe 10	consumidor.exe	10 5 16 8 4 2 1	7
3	produtor.exe 13	consumidor.exe	13 40 20 10 5 16 8 4 2 1	10

Esses valores foram determinados pelas regras de Collatz e servem como referência; não são saídas observadas dos executáveis. Em todos os casos espera-se que o produtor informe a quantidade gravada, aguarde o consumidor e confirme a leitura. O consumidor deve informar a mesma quantidade e a sequência correspondente.

5.4 Casos adicionais de teste

Além dos três exemplos obrigatórios, devem ser verificados: entrada 1; rejeição de zero, negativos, texto e decimais; argumentos ausentes ou em excesso; entrada acima do limite de `uint64_t`; overflow de $3n+1$; segundo produtor simultâneo; e consumidor iniciado sem produtor. Essas verificações de execução permanecem pendentes pelo bloqueio descrito na seção 5.2.

6. Respostas às questões conceituais

1. O que é memória virtual e qual problema ela procura solucionar?

É a abstração que permite ao processo usar endereços independentes da posição física dos dados. Facilita proteção, isolamento e alocação, e permite executar programas sem exigir que todo o conteúdo virtual esteja simultaneamente na RAM.

2. O que é um espaço de endereçamento virtual?

É o conjunto de endereços virtuais utilizáveis por um processo. Nele podem existir regiões de código, pilha, heap, bibliotecas e mapeamentos. Um endereço desse conjunto não identifica diretamente uma posição física.

3. Por que dois processos possuem, normalmente, espaços independentes?

Para que cada processo administre seus dados sem alterar acidentalmente os dados de outro. O isolamento também permite que o sistema controle permissões e limite as consequências de acessos inválidos.

4. Por que o consumidor não pode simplesmente acessar `int valor = 10` do produtor?

A variável ocupa uma região privada do produtor. Seu endereço só é interpretado no contexto daquele espaço virtual. O mesmo número de endereço no consumidor pode identificar outros dados ou uma região não mapeada. O valor precisa ser comunicado por um mecanismo de IPC.

5. Qual é a função da memória compartilhada nesta atividade?

Transportar a sequência e sua quantidade do produtor ao consumidor. O produtor grava no bloco mapeado e o consumidor acessa esse mesmo armazenamento por sua própria vista.

6. Qual é a diferença entre memória privada e compartilhada?

A privada pertence ao espaço de um processo e não fica diretamente disponível aos demais. A compartilhada é uma região explicitamente disponibilizada a vários processos, com permissões definidas. Seu acesso exige coordenação quando há escrita.

7. Qual é a função de `CreateFileMapping()` ?

Criar ou abrir um objeto de mapeamento. Nesta implementação, o produtor solicita um objeto nomeado de 32.776 bytes com proteção de leitura e escrita e respaldo no arquivo de paginação. A criação do objeto ainda não fornece o ponteiro para acessá-lo.

8. Qual é a função de `MapViewOfFile()` ?

Mapear uma vista do objeto no espaço virtual do processo chamador e devolver o endereço para acessar seus bytes. O produtor usa uma vista com escrita; o consumidor usa somente leitura.

9. Por que o consumidor também precisa mapear a região?

Porque o handle obtido ao abrir o objeto não é um ponteiro para os dados. O consumidor precisa de uma vista em seu próprio espaço de endereçamento para conseguir ler a estrutura.

10. Qual é a relação entre memória virtual e esse mapeamento?

O mapeamento associa uma região virtual de cada processo ao mesmo armazenamento compartilhado. A tradução gerenciada pelo sistema preserva a independência dos processos e torna aquela região acessível a ambos.

11. Por que produtor e consumidor precisam ser sincronizados?

Porque podem executar em momentos e velocidades diferentes. O consumidor só deve interpretar o bloco depois de todas as escritas do produtor estarem concluídas e publicadas.

12. O que pode acontecer se a leitura for antecipada?

O consumidor pode observar quantidade ainda não publicada, zeros iniciais ou parte da sequência. Em um protocolo mal definido, uma quantidade incoerente pode até provocar leitura fora dos limites. A espera e a validação da quantidade evitam esses problemas nesta implementação.

13. Qual mecanismo de sincronização foi utilizado?

Dois eventos nomeados de reset manual. O produtor sinaliza `Pronto` após escrever; o consumidor aguarda esse evento, lê e sinaliza `Lido`. O produtor aguarda `Lido` antes de encerrar. Cada rodada cria objetos novos, inicialmente não sinalizados.

14. Qual é a função de `WaitForSingleObject()` na solução?

Suspender a thread até o evento ser sinalizado ou até ocorrer timeout. O consumidor espera os dados e o produtor espera a confirmação. O código distingue sucesso, timeout e falha, sem usar laços de espera ocupada [5].

15. Como endereços virtuais diferentes acessam a mesma região?

Cada processo possui sua própria tradução de endereços. A página que contém o endereço A no produtor e a página que contém o endereço B no consumidor podem ser associadas ao mesmo armazenamento do objeto compartilhado. O que precisa ser comum é o objeto e o deslocamento dos dados, não o valor numérico do ponteiro.

7. Questão central e conclusão

Dois processos independentes acessam os mesmos dados quando o sistema operacional mapeia o mesmo objeto compartilhado nos respectivos espaços virtuais. A independência das regiões privadas permanece; apenas o bloco disponibilizado participa da comunicação. Os eventos organizam a publicação e a leitura desse bloco.

A implementação reúne geração de Collatz, memória compartilhada nomeada, sincronização e liberação de recursos. A compilação foi verificada. A validação prática da comunicação permanece pendente porque o Windows bloqueou os executáveis; os três testes obrigatórios devem ser realizados e documentados antes da entrega final em sala.

8. Referências consultadas

- [1] Microsoft Learn. Virtual Address Space. <https://learn.microsoft.com/en-us/windows/win32/memory/virtual-address-space>
- [2] Microsoft Learn. CreateFileMappingW function. <https://learn.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-createfilemappingw>
- [3] Microsoft Learn. Event Objects. <https://learn.microsoft.com/en-us/windows/win32/sync/event-objects>
- [4] Microsoft Learn. Synchronization and Multiprocessor Issues. <https://learn.microsoft.com/en-us/windows/win32/sync/synchronization-and-multiprocessor-issues>
- [5] Microsoft Learn. WaitForSingleObject function. <https://learn.microsoft.com/en-us/windows/win32/api/synchapi/nf-synchapi-waitforsingleobject>
- [6] Microsoft Learn. Creating Named Shared Memory. <https://learn.microsoft.com/en-us/windows/win32/memory/creating-named-shared-memory>

Enunciado: Atividade Complementar — Memória Virtual, Memória Compartilhada e Comunicação entre Processos, fornecido na disciplina. Consulta à documentação em 01/10/2026. A literatura de Silberschatz indicada no enunciado é leitura complementar recomendada; não foi consultada diretamente nesta resolução.